Iterating

The Enumerable module uses each as the building block for basically all of its functionality, starting with the methods in this section that are more elaborate versions of iteration. The Enumerable module doesn't define each; instead, it depends on any class that includes Enumerable to define each. The basic contract of each is that it accepts a block and yields each element of the container in turn to that block. For most Enumerable clients that's straightforward—an Array or Set yields each element in the container in order. Hash#each is slightly different; it yields each key/value pair as a two-element array in order.

Unless otherwise specified, every method in this section that take a block has an alternate form that does not take a block and returns an Enumerator—we'll talk about that form in Enumerator, on page 544.

There are a handful of methods that are just slightly different structures of each, starting with reverse_each, where foo.reverse_each is equivalent to foo.reverse.each, but the single method version is a little faster because it doesn't create an intermediate structure.

The default each calls its block with a single element of the enumerable.

If you want to call each element through the array more then once, you can use cycle, which takes one argument and behaves as that argument amount of successive calls to each:

```
result = []
arr = ["a", "b", "c"]
arr.cycle(3) { |x| result << x } # => nil
result                           # => ["a", "b", "c", "a", "b", "c", "a", "b",
                                 # .. "c"]
```

If you want to call a block with multiple successive elements of the data structure, you can do that with each_slice or each_cons. Both of these methods take an argument that is the number of elements that each block call gets. The methods differ in how they generate the list of successive elements.

The each_slice method yields non-overlapping lists. The second call starts with the element after the end of the first call. The each_cons method yields overlapping lists, the second call starts with the second element of the first call:

```
elements = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

puts "each_slice"
elements.each_slice(3) { |x, y, z| puts "#{x} #{y} #{z}" }

puts
puts "each_cons"
elements.each_cons(3) { |x, y, z| puts "#{x} #{y} #{z}" }
```

produces:

```
each_slice
1 2 3
4 5 6
7 8 9
10

each_cons
1 2 3
2 3 4
```